

Convenciones y buenas prácticas en Java

Las convenciones de código existen para permitir un adecuado mantenimiento del mismo y para mejorar la lectura del código permitiendo entender código nuevo mucho más rápido y a fondo. Para que las convenciones funcionen, cada programador debe tratar de ser lo más fiel posible a estas.

Uso de import

Sólo importaremos las clases requeridas, nunca paquetes completos salvo indicación expresa de lo contrario:

<pre>/* Correcto */ import java.awt.Frame; import java.awt.Graphics;</pre>	<pre>/* Incorrecto */ import java.awt.*;</pre>
--	--

Declaraciones de variables

- Tipo de datos necesario y requerido, adecuado al dato a almacenar.
- Las variables locales se deben inicializar en el momento de declararlas o justo después, a menos que no requieran valor inicial salvo el valor por defecto.
- Los arrays tienen los corchetes [] unidos a su tipo de datos:

```
String[] nombres; //correcto  
String nombres[]; //incorrecto
```

Nombres de identificadores

Para los identificadores debemos usar el alfabeto anglosajón y números de la tabla ASCII. No se deben usar caracteres con tilde ni otros como (ñ,ç), entre otros ejemplos. Las barras bajas o guiones tampoco se usan. **Los nombres de los identificadores deben ser siempre lo más descriptivos posible, ya sea variable, método o clase.** Solo se usan identificadores de un solo carácter para representar los contadores en bucles:

- Nombre de las **clases o interfaces**: *UpperCamelCase*.
- Nombre de los **métodos**: *lowerCamelCase*. Suelen ser verbos o frases.
- Nombres de **constantes**: *CONSTANT_CASE*: todo en mayúsculas, separando con barra baja.
- **Variables locales, atributos de clase, nombres de parámetros**: *lowerCamelCase*.

Convenciones de nomenclatura

- **Autoexplicativo**: un nombre debe revelar su intención para que todos puedan entender y cambiar el código fácilmente. Por ejemplo, las variables llamadas “d” o “str” no revelan nada; sin embargo, los nombres “daysToExpire” o “edad” revelan claramente su intención. Ten en cuenta que, si un nombre requiere un comentario para describirse a sí mismo, entonces el nombre no se explica por sí mismo.

- **Distinciones significativas:** si los nombres deben ser diferentes, también deberían significar algo diferente. Por ejemplo, las variables “a1” y “a2” son distinciones sin sentido; y las variables “fuente” y “destino” son distinciones significativas.

Estructura del código

- Una sola instrucción por línea.
- Las líneas de código no deben superar los 80/100 caracteres. Si no, se deben **romper** antes de algún operador.
- Si la declaración del método es demasiado larga, o una expresión aritmética es demasiado larga, o en una sentencia *if*, debo romper.
- Si una operación aritmética o lógica se compone de distintos tipos de operaciones con distinta jerarquía, se deben usar paréntesis para facilitar su legibilidad.

```
public void ejecutarAccion(
    TipoParametro parametro1, TipoParametro parametro2,
    TipoParametro parametro3){
    ...
}

if ((condicion1 && condicion2)
    || (condicion3 && condicion4)
    || !(condicion5 && condicion6)) {
    llamarMetodo();
}

longName1 = longName2 * (longName3 + longName4 - longName5)
+ (4 * longname6);
//Siempre con el operador al principio de línea
```

- Los espacios en blanco mejoran la legibilidad. Se deben colocar entre operadores, después de los puntos y coma de los bucles for, después de los operadores de asignación, etc.

```
cantidadTotal = cantidadInicial + cantidadFinal;

for(int i = 0; i < cantidadTotal; i++){
    ...
}

public String getItem(int fila, int columna) {
    ...
}

getItem(cantidadInicial, cantidadFinal);
```

- **Comparación de objetos:** recordar que tanto los *Strings* como las tipos *Wrapper* son objetos y sus variables solo contienen sus referencias (direcciones). **Los objetos no se comparan con ==.**

- **Tipos primitivos mejor que clases *wrapper* (envoltorio):** Las clases wrapper, al ser objetos, proveen de métodos para trabajar mejor con ellas, pero al igual que los Strings son más lentos que los tipos primitivos. Es decir, mejor “int” que “Integer”.

```
int x = 10;
int y = 10;
```

```
Integer x1 = new Integer(10); //Mal
Integer y1 = new Integer(10); //Mal
```

- Evitar la creación **innecesaria** de variables y objetos. Como se ha dicho, generan mucha carga de procesamiento.
- **Usar siempre una variable para un único propósito:** a veces sentimos la tentación de reutilizar una variable, pero complica la legibilidad.

```
...
int resultadoTotal = resultadoInicial - resultadoFinal;
...
```

- **Bucle for:** optar por el *for* siempre que se pueda (frente a while, do-while) salvo que no se sepa con antelación las iteraciones que se van a realizar. Las ventajas son que reúne todo el control del bucle en la misma línea (inicio, fin, e incremento), y la variable de control no es accesible desde fuera de él. Si se necesita modificar su variable de control, hay que usar otro bucle.
- **Constantes:** nunca definir nuevas constantes que ya proporcione Java, como el número pi en **Math.PI**.
- **Switch:** debe llevar el caso default.
- **Indentación:** el sangrado (también conocido como *indentación*) deberá aplicarse a toda estructura que esté **lógicamente contenida dentro de otra**. El sangrado será de un tabulador. Puedes imaginarlo como la estructura jerárquica que siguen las carpetas en forma de árbol.
- **Optimizaciones:**
 - Nunca uses 2 bucles si puedes resolver el problema con 1, y esto es extensible a otras estructuras.
 - El anidamiento de bucles debe usarse con moderación y siempre que no haya otra solución mejor. Trata de evitarlos.
 - **Simplifica** al máximo: trata de evitar la construcción de código complejo siempre que sea posible. Así, encontrarás menos bugs y ahorrarás tiempo en solucionar errores. Tu objetivo debería ser el de escribir código funcional, sin filigranas.
 - Optimiza expresiones condicionales o de otro tipo: escríbelas lo más compactas posibles y empleando los operadores de forma eficiente, es decir:

```
/* Correcto */
if (x >= 0) ...
...
resultado = y + 2;
```

```
/* Incorrecto */
if (x > 0 || x == 0)...
...
x = y + 2;
resultado = x;
```